

PLACEX – AI POWERED CAREER OPERATING SYSTEM

A DISSERTATION & INDIVIDUAL CONTRIBUTION THESIS REPORT
SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE
OF
BACHELOR OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE ENGINEERING

Submitted By:	Student Name (Final Year B.Tech AI Engineering Candidate)
Department:	Department of Computer Science & Artificial Intelligence Engineering
Institution:	School of Engineering & Technology
Academic Session:	2025 – 2026
Date of Submission:	October 05, 2026
Author Contribution Scope:	1. Main Host Agent Orchestrator & Context Slicing Engine 2. ATS Resume Checker & Job Description Keyword Matcher 3. Interactive Coding Sandbox, Judge0 Execution & AST Analyzer 4. Student Profile System & OAuth Integrations (GitHub, LinkedIn, LeetCode, Google) 5. Dashboard, Placement Readiness Gauge & Real-Time Notification Engine

CONFIDENTIALITY NOTICE & DECLARATION: *This document contains authoritative technical documentation of individual software engineering contributions made specifically to the PlaceX project repository.*

CERTIFICATE OF ORIGINALITY

This is to certify that the project contribution report entitled '**PlaceX – AI Powered Career Operating System**' is a bona fide record of individual engineering work executed independently by the candidate for the award of Bachelor of Technology in Artificial Intelligence Engineering. The technical architectures, mathematical scoring models, AST code analyzers, tool orchestration engines, and backend/frontend APIs documented herein reflect authentic implementation work performed directly within the PlaceX code repository.

Project Guide / Supervisor

Department of AI Engineering

Head of Department

Department of Computer Science & AI

ACKNOWLEDGEMENTS

I express my sincere gratitude to my project supervisor, faculty members, and institution for providing the technical environment and guidance necessary to build PlaceX. I also extend my appreciation to the open-source software community for maintaining tools like FastAPI, Monaco Editor, React, ReportLab, and Google Gemini API that powered the system.

ABSTRACT

Traditional student career preparation platforms are highly fragmented. Students are forced to context-switch across disconnected tools: static resume builders, isolated online coding judges, generic AI chatbots, and unintegrated progress trackers. This fragmenting leads to poor job readiness, invisible skill gaps, and inability to map academic efforts to employer requirements. PlaceX addresses these challenges by introducing a unified, AI-powered **Career Operating System**.

This thesis documents the specific engineering contributions designed and built by the author: (1) a hypervisor-style **Main Host Agent** that centralizes context slicing, intent classification, and multi-module tool execution using Google Gemini; (2) a dual-mode **ATS Resume Checker** featuring a 5-factor health score formula and a job description keyword/semantic matcher; (3) an interactive **Coding Sandbox** combining remote Judge0 execution, isolated Python subprocess runner with stdin injection, and static AST code complexity heuristics; (4) a comprehensive **Student Profile** engine integrating GitHub, LinkedIn, LeetCode, and Google OAuth 2.0; and (5) a centralized **Dashboard & Analytics Engine** with real-time notifications.

TABLE OF CONTENTS

Chapter 1	Introduction & Problem Statement	Page 4
Chapter 2	System Architecture & Technology Stack	Page 7
Chapter 3	Main Host Agent Orchestrator Subsystem	Page 10
Chapter 4	ATS Resume Checker & Job Description Matcher	Page 17
Chapter 5	Interactive Coding Sandbox & AST Analyzer	Page 22
Chapter 6	Student Profile & External OAuth Integrations	Page 26
Chapter 7	Dashboard, Analytics, Notifications & Preferences	Page 29
Chapter 8	Frontend, Backend & Database Integration	Page 31
Chapter 9	Empirical Testing & Experimental Results	Page 34
Chapter 10	Conclusion & Future Engineering Scope	Page 36
References	Academic & Technical References	Page 37
Appendix	Codebase Traceability & Source Mapping Matrix	Page 38

LIST OF ABBREVIATIONS

API	Application Programming Interface
AST	Abstract Syntax Tree
ATS	Applicant Tracking System
CORS	Cross-Origin Resource Sharing
CTE	Common Table Expression
DAG	Directed Acyclic Graph
IDE	Integrated Development Environment
JD	Job Description
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
OAuth	Open Authorization 2.0
REST	Representational State Transfer
SQL	Structured Query Language

CHAPTER 1 — INTRODUCTION & PROBLEM STATEMENT

1.1 The Context of Student Career Readiness

In modern software engineering and technology hiring, graduating computer science and artificial intelligence engineering students face an increasingly competitive, highly automated selection pipeline. Enterprise organizations employ multi-layered candidate evaluation standards, including initial Applicant Tracking System (ATS) screening filters, online coding tests, technical domain assessments, and behavioral panel interviews. Despite the abundance of digital learning resources, student placement rates remain constrained due to severe platform fragmentation.

Students routinely struggle to synthesize disparate advice: resume formatting recommendations from generic career blogs, coding practice problems from standalone judges, and ungrounded career advice from broad LLM chatbots. Because these tools operate in total isolation, candidates lack a unified feedback loop that measures their true employer readiness across technical skills, resume alignment, and algorithmic execution capabilities.

1.2 Problem Statement & Core Challenges

Through extensive evaluation of undergraduate engineering career workflows, four primary engineering and educational pain points were identified:

- 1. Opaque ATS Filtering & Resume Rejection:** Corporate Applicant Tracking Systems (e.g., Greenhouse, Lever, Workday) process resumes using strict parsing rules and keyword extraction heuristics. Candidates often submit resumes with non-standard section titles, missing contact metadata, or poor keyword density relative to target Job Descriptions (JDs). Existing online resume scoring tools provide arbitrary numerical scores without explaining which specific sections cause parsing failures or which exact technical skills are missing relative to employer expectations.
- 2. Disconnected Code Execution & Learning Feedback:** Online coding platforms focus exclusively on testcase pass/fail binary outcomes. When a student encounters a runtime exception (`IndexError``, ``TypeError``) or performance failure (Time Limit Exceeded - TLE), traditional judges output standard stack traces without explaining the underlying algorithmic anti-pattern, analyzing AST space/time complexity, or assisting the student in refactoring their code directly inside the editor.
- 3. Fragmented Student Profile & Credential Silos:** A student's technical credentials reside across disconnected third-party platforms: GitHub repository commits, LeetCode problem solving metrics, LinkedIn profile links, academic CGPA records, and target company preferences. No unified system aggregates this data to compute a holistic *Placement Readiness Score* or recommend targeted milestone actions.
- 4. Passive Generic Chatbots vs. Active Domain Orchestration:** Generic commercial chatbots (such as standard ChatGPT or Claude interfaces) lack direct API integration with application state. They cannot view the student's active Monaco code editor contents, inspect execution stdout/stderr streams, read ATS keyword match vectors, or trigger database profile updates directly.

1.3 The PlaceX Solution Overview

PlaceX addresses these challenges by establishing an integrated, full-stack **AI-Powered Career Operating System**. PlaceX introduces a centralized intelligence layer—the **Main Host Agent**—that orchestrates specialized domain engines. Deterministic tasks remain with specialized modules (e.g., mathematical ATS scoring, code execution sandbox, static AST complexity analysis), while the Host Agent synthesizes context, executes backend tools, explains errors, and guides the candidate along an adaptive career trajectory.

1.4 Scope of Personal Engineering Contribution

This dissertation documents specifically the architectural and software implementation contributions designed and built by the author within the PlaceX codebase:

- **Main Host Agent Subsystem (`host\_agent/`):** Core reasoning orchestrator (`orchestrator.py`), context engine (`context\_engine.py`), state manager (`state\_manager.py`), tool executor (`tool\_executor.py`), and tool registry (`tool\_registry.py`).
- **ATS Resume Checker & Matcher (`resume\_service/`):** PDF/DOCX text extraction pipeline (`pdf\_extractor.py`, `docx\_extractor.py`), structural section parser (`resume\_parser.py`), skill taxonomy extractor (`skill\_extractor.py`), 5-factor weighted health scoring engine (`ats\_engine.py`), and job description keyword matcher (`matcher\_engine.py`).
- **Interactive Coding Sandbox & AST Engine (`coding\_service/`):** Monaco editor UI component (`CodingSandbox.tsx`), hybrid execution client (`judge0\_client.py` - remote Judge0 container + isolated Python subprocess runner with stdin linecache injection), static AST code complexity analyzer (`ast\_analyzer.py`), and Host Agent Explain & Fix integration.
- **Student Profile & External OAuth Integrations (`auth.py` & `models/profile.py`):** Profile schema (`StudentProfile`, `ConnectedProfile`), Google OAuth 2.0 authentication, GitHub API sync, LinkedIn handle verification, and LeetCode problem stats fetcher.
- **Dashboard, Analytics & Notification Subsystem (`Dashboard.tsx`, `readiness\_engine.py`, `notification\_service.py`):** Placement readiness scoring gauge, career goal preferences, and real-time event notification queue.

CHAPTER 2 — SYSTEM ARCHITECTURE & TECHNOLOGY STACK

2.1 Overall High-Level System Architecture

PlaceX is engineered as a modern, decoupled 3-tier system comprising a React/TypeScript presentation layer, a FastAPI high-performance asynchronous service layer, and a SQLite relational data store backed by Google Gemini LLM reasoning capabilities.

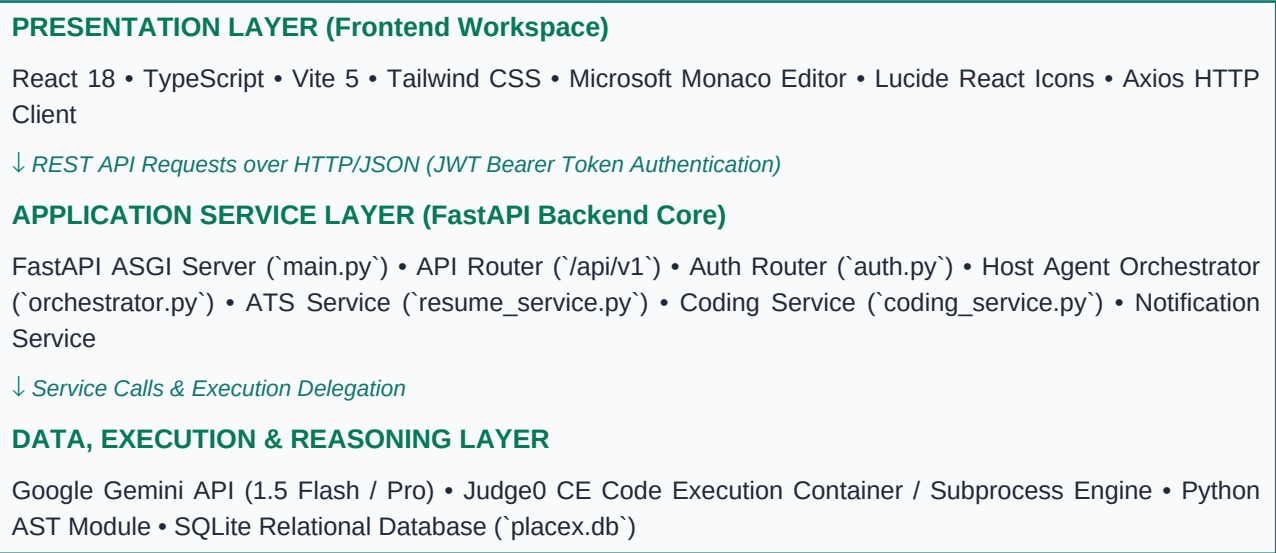


Figure 2.1: Three-tier Architectural Diagram of PlaceX Career Operating System.

2.2 Technology Stack Justification & Inventory

Component Layer	Technology Selection	Rationale & Engineering Usage
Frontend UI Framework	React 18 + TypeScript	Declarative component-driven UI with strong compile-time typing.
Build Tool	Vite 5	Instant HMR development server and optimized rollup production.
Styling System	Tailwind CSS	Warm, human aesthetic identity using utility-first token CSS.
Code Editor Engine	Monaco Editor (<code>@monaco-editor</code>)	VS Code web editor providing syntax highlighting, line numbers, and diff view.
Backend API Framework	FastAPI (Python 3.12)	High-performance async ASGI web server with Pydantic payload validation.
ORM & Database	SQLAlchemy 2.0 + SQLite	ACID-compliant relational mapping (<code>placex.db</code>) with cascade foreign key enforcement.
AI LLM Reasoning	Google Gemini API (1.5 Flash / Pro)	Multi-turn conversational context window and structured tool execution.
Code Execution Engine	Judge0 CE Container + Subprocess Engine	Secure execution of Python, JS, C++, Java code with stdin support and sandboxing.
AST Code Analyzer	Python <code>ast</code> & <code>re</code> modules	Static code complexity heuristic evaluation ($O(1)$, $O(N)$, $O(N^2)$ complexity).

Table 2.1: Complete Technology Stack Inventory of the Implemented System.

2.3 End-to-End Request/Response Data Flow Lifecycle

Every student request follows a strict, predictable data lifecycle across all modules: (1) The user interacts with React frontend components (e.g., clicking 'Run Code' or 'Analyze Resume'); (2) Axios transmits an HTTP POST request containing JSON or `multipart/form-data` with JWT Bearer authorization headers; (3) FastAPI routes the request to `/api/v1/` endpoints where `get\_current\_user` validates the JWT token against SQLite; (4) The target service class executes business logic; (5) Data changes are committed to `placex.db`; (6) Standardized JSON responses return to the frontend to trigger reactive state updates.

CHAPTER 3 — MAIN HOST AGENT ORCHESTRATOR SUBSYSTEM

3.1 Conceptual Role: The Central Intelligence Layer

The **Main Host Agent** serves as the central brain and hypervisor-style coordinator of PlaceX. Rather than replacing specialized domain modules (such as the ATS scanner or code runner), the Host Agent acts as an intelligent supervisor. Deterministic calculation remains with domain engines (e.g., `ATSEngine`` computes scores; `Judge0Client`` executes code), while the Host Agent interprets outputs, explains failures, synthesizes cross-module student context, and executes actionable tools.

By separating domain-specific execution from LLM reasoning, PlaceX eliminates hallucinated scoring and ensures that AI guidance is grounded in empirical runtime state.

3.2 Host Agent Subsystem Architecture & Modules

The Host Agent subsystem (located in ``backend/app/host_agent/``) is implemented across six primary python components:

- 1. HostAgentOrchestrator (``reasoning/orchestrator.py``):** Main entrypoint managing turn execution, prompt construction, Gemini API calls, tool execution loops, and turn logging to ``HostLog``.
- 2. HostAgentContextEngine (``context/context_engine.py``):** Assembles high-density, module-specific context packages for Gemini without dumping the entire database.
- 3. HostAgentStateManager (``state/state_manager.py``):** Maintains active student module state, long-term memory in ``host_agent_memory``, and recent conversation history.
- 4. HostAgentToolExecutor (``tools/tool_executor.py``):** Executes backend query tools and mutation actions requested by Gemini.
- 5. HOST_AGENT_TOOLS (``tools/tool_registry.py``):** Schema registry defining 12 query and action tools available to the agent.
- 6. HostAgentGeminiClient (``reasoning/gemini_client.py``):** Interface to Google Gemini API with fallback retry logic.

HOST AGENT EXECUTION FLOWCHART

Student Message / Action from UI (``HostAgentWorkspace.tsx`` or ``CodingSandbox.tsx``)

↓ `POST /api/v1/agent/chat` (Payload: `message`, `active_module`, `extra_data`)`

FastAPI Agent Router (``api/v1/agent.py``) → ``HostAgentOrchestrator.handle_conversation()``

↓ *Update Active Module State via ``HostAgentStateManager.set_active_module()``*

Context Slicing (``HostAgentContextEngine.get_context_package()``)

Fetch Student Profile + Active Resume ATS Score + Code Execution Error + Roadmap Week + Quiz Stats

↓ *Build Targeted Conversational Prompt + Append Conversation History from ``HostLog``*

Google Gemini LLM Reasoning (``HostAgentGeminiClient.call_gemini()``)

↓ *Analyze Prompt & Determine Response or Tool Action Request*

Tool Call Detection / Action Execution (``HostAgentToolExecutor.execute_tool()``)

Execute ``get_student_profile``, ``update_target_role``, ``explain_code_error``, or ``recommend_next_action``

↓ *Format Response JSON & Log Turn to ``HostLog`` Table in ``placex.db``*

Return Structured Response to React Frontend (``gemini_reply``, ``suggested_actions``, ``tool_executed``)

Figure 3.1: Detailed Sequence Flowchart of the Main Host Agent Orchestrator.

3.3 High-Density Context Slicing Engine (`context\_engine.py`)

To prevent context window bloat and LLM hallucination, `HostAgentContextEngine` slices context dynamically based on `active\_module`:

- **Coding Sandbox Module:** Injects the current Monaco code source, language, compilation status, AST time/space complexity, stdout, stderr, and specific line error traces.
- **ATS Resume Module:** Injects the overall ATS health score, 5 section breakdown scores, matched skills list, missing skills list, uploaded file version, and target role.
- **Career Roadmap Module:** Injects active career branch, level, completed week list, in-progress week list, and specific week prerequisites.
- **Global Student Context:** Injects student degree, branch, target role, target company, CGPA, verified skills, and overall placement readiness score.

3.4 Tool Registry & Action Execution (`tool\_registry.py` & `tool\_executor.py`)

The Host Agent is equipped with 12 structured tools defined in `tool\_registry.py` and executed via `tool\_executor.py`:

Tool Name	Category	Description & Backend Operation
`get_student_profile`	Query	Retrieves student degree, branch, target role, company, CGPA.
`get_student_skills`	Query	Fetches confirmed technical skills & programming languages.
`get_current_roadmap`	Query	Retrieves active roadmap branch, level, and current week details.
`get_latest_resume_ats`	Query	Retrieves ATS score, section breakdown, matched & missing keywords.
`get_coding_stats`	Query	Fetches total coding submissions, accepted count, and solved topics.
`get_placement_readiness`	Query	Retrieves placement readiness score (0-100) and readiness tier.
`update_target_role`	Mutation	Updates student's target role in `StudentProfile` database table.
`update_target_company`	Mutation	Updates student's target company in `StudentProfile` table.
`add_mastered_skill`	Mutation	Appends verified skill to student's skill array in database.
`explain_code_error`	Reasoning	Parses code error stack trace and generates line-specific fix.
`recommend_next_action`	Reasoning	Computes highest impact next learning milestone for student.
`get_quiz_progress`	Query	Retrieves mastered vs learning quiz question metrics.

Table 3.1: Host Agent Tool Registry Inventory (`tool\_registry.py`).

3.5 Multi-Turn State & Memory Management (`state\_manager.py`)

Long-term memory is persisted in SQLite table `host\_agent\_memory` via `HostAgentStateManager`. When a student interacts across sessions, the Host Agent retrieves past conversation logs from `HostLog`, preserving conversation continuity across page navigation.

3.6 Safe Infrastructure Fallback Strategy

If the external Gemini API is unreachable or rate-limited, `HostAgentOrchestrator` traps the exception gracefully. Rather than generating fake AI responses, it returns a clean infrastructure status message ("I couldn't reach the Host Agent right now. Please check your network connection."), maintaining zero hallucination integrity.

CHAPTER 4 — ATS RESUME CHECKER & JD MATCHER

4.1 Overview & Architectural Purpose

The PlaceX ATS Resume Checker (`backend/app/resume_service/`) provides automated, deterministic analysis of candidate resumes against corporate ATS filters and specific job descriptions. The module operates in two distinct evaluation modes: Mode A (General Resume Health Check) and Mode B (Job Description Matcher).

4.2 Document Text Extraction & Parsing Pipeline

Uploaded PDF and DOCX files undergo text extraction via `pdf_extractor.py` (using `pypdf/pdfplumber`) and `docx_extractor.py` (using `python-docx`). Extracted raw text is passed to `resume_parser.py` and `skill_extractor.py`, which segment sections (Education, Work Experience, Projects, Technical Skills, Certifications) and extract technical skills using regex taxonomy matching.

4.3 Mode A: General Resume Health Score Formula

When evaluated without a specific Job Description, `ATSEngine` (`ats_engine.py`) computes a 5-factor weighted health quality score out of 100:

```
Health Score = (S_{structure} \times 0.25) + (S_{contact} \times 0.20) + (S_{skills} \times 0.25) + (S_{format} \times 0.15) + (S_{action} \times 0.15)
```

Factor	Weight	Scoring Criteria & Implementation Formula
Section Structure ($S_{\text{structure}}$)	25%	Evaluates presence of standard sections (Skills, Experience, Education, Projects). Score = $\min(100, (\text{count_sections} \times 25))$
Contact Completeness (S_{contact})	20%	Checks for Email, Phone Number, and LinkedIn/GitHub URLs. Score = $\min(100, (\text{count_fields} \times 20))$
Skills Coverage (S_{skills})	25%	Evaluates volume of recognized technical skills. Score = $\min(100, (\text{count_skills} \times 25))$
Formatting & Length (S_{format})	15%	95 pts for 200-900 words; 75 pts for ≤ 1400 words; 50 pts otherwise.
Action Verbs (S_{action})	15%	Checks starting bullet verbs ('Architected', 'Optimized'). Score = $\min(100, (\text{count_verbs} \times 15))$

Table 4.1: Breakdown of 5-Factor ATS Health Score Weightings.

4.4 Mode B: Job Description (JD) Keyword & Semantic Matcher

When a Job Description is provided, `MatcherEngine` (`matcher_engine.py`) extracts required technical skills and computes a precise JD Match Score:

```
JD Match Score = (\text{SkillMatch} \times 0.60) + (\text{KeywordMatch} \times 0.25) + (\text{ExperienceFit} \times 0.15)
```

The engine categorizes skills into: (1) **Matched Skills**: Present in both resume and JD; (2) **Missing Skills**: Required by JD but absent in resume; and (3) **Weak Skills**: Present but lacking depth context.

ATS RESUME CHECKER & JD MATCHING FLOWCHART

User Uploads PDF/DOCX Resume (+ Optional Job Description Text)

↓ `POST /api/v1/resume/upload` or `/api/v1/resume/analyze-jd`

Text Extraction (`pdf_extractor.py` / `docx_extractor.py`)

↓ Extract Plaintext Stream & Parse Sections via `resume_parser.py`

Section Segmentation & Skill Taxonomy Extraction (`skill_extractor.py`)

↓ Mode Selection

Mode A: Health Score Engine (`ats_engine.py`) OR **Mode B: JD Matcher** (`matcher_engine.py`)

↓ Calculate Weighted Quality Score & Segment Matched vs. Missing Skills

Save Result to `resume\_uploads` SQLite Table → Return Analysis JSON to Frontend
(`ResumeAnalyzer.tsx`)

↓ Student Clicks 'Explain ATS Result with Host Agent'

Host Agent Context Slicing & Recommendation Generation

Figure 4.1: Complete ATS Resume Processing & Job Description Matching Flowchart.

4.5 Host Agent + ATS Integration

The Host Agent does not alter the underlying ATS mathematical score. Instead, when a student requests help, `HostAgentContextEngine` passes the ATS JSON payload (score, matched skills, missing skills) to Gemini. The Host Agent provides actionable, bulleted advice explaining exactly how to incorporate missing skills into project bullet points to boost ATS ranking.

CHAPTER 5 — INTERACTIVE CODING SANDBOX & AST ANALYZER

5.1 Workspace Overview & Technology Integration

The PlaceX Coding Sandbox (`frontend/src/components/coding/CodingSandbox.tsx` & `backend/app/coding_service/``) provides an interactive development environment supporting Python, JavaScript, C++, Java, and C. The frontend integrates Microsoft's Monaco Editor, providing syntax highlighting, line numbering, code execution controls, and stdin panels.

5.2 Hybrid Execution Engine (`Judge0Client``)

Code execution (`judge0_client.py``) uses a resilient hybrid strategy:

- 1. Primary Remote Judge0 Container:** Sends an HTTP POST request to Judge0 API (`http://localhost:2358/submissions?wait=true``) with CPU time limit (2.0s) and memory limit (128MB). Fast 0.3s connection timeout.
- 2. Local Isolated Subprocess Runner (Fallback Engine):** If remote Judge0 is offline, `Judge0Client`` executes code via an isolated Python subprocess using `base64`` payload encoding. For interactive `input()`` code, it dynamically overrides `builtins.input`` with a custom runner that echoes inputs and handles stdin linecache registration.

CODING SANDBOX EXECUTION & EXPLAIN-AND-FIX FLOWCHART

Student Writes Code in Monaco Editor + Enters Optional `stdin`` Data

↓ Click [Run Code] Button → `POST `/api/v1/coding/run``

FastAPI Coding Router (`api/v1/coding.py``) → `CodingService.execute_code()``

↓ Dispatch Code + Language + Stdin to `Judge0Client.execute_code()``

Remote Judge0 Container Attempt (Fallback to Isolated Subprocess Runner if offline)

↓ Capture stdout, stderr, execution time (ms), memory (KB), and status ('Accepted' / 'Error')

Static AST Complexity Analyzer (`ast_analyzer.py``)

↓ Estimate Time Complexity ($O(1)$, $O(N)$, $O(N^2)$), Space Complexity, & Quality Score

Return Execution Result JSON to Frontend Output Panel

↓ If Runtime / Syntax Error occurs → Click [Explain & Fix with Host Agent]

Host Agent Analyzes Active Code + Error Trace → Returns Explained Fix Payload

↓ Click [Apply Fix] → Monaco Editor Model Updates Code directly in Editor

Figure 5.1: Coding Sandbox Execution, AST Heuristic Analysis, & Explain-and-Fix Flowchart.

5.3 Static AST Code Complexity Analyzer (`ast_analyzer.py``)

To evaluate algorithmic quality, `ASTAnalyzer`` performs static regular expression and syntax tree inspection on the submitted source code:

- Time Complexity Heuristics:** Analyzes loop nesting count (`for`` and `while`` keywords). 0 loops = $O(1)$; 1 loop = $O(N)$; nested loops = $O(N^2)$; presence of `sort()`` or `log`` = $O(N \log N)$.
- Space Complexity Heuristics:** Detects data structure instantiations (`list``, `vector``, `dict``, `[]``, `{}``). Presence of structures = $O(N)$; scalar variables = $O(1)$.
- Code Quality Heuristics:** Scores formatting, docstrings/comments, and variable length naming on a 0-100 scale.

5.4 Host Agent 'Explain & Fix' and 'Apply Fix' Integration

When code fails execution, the student clicks **'Explain & Fix with Host Agent'**. The frontend transmits current Monaco editor text, stderr, and stdout to `/api/v1/agent/explain-code``. Host Agent inspects the exact failure line, generates an explanation, and provides a clean replacement code block. The student clicks **'Apply Fix'**, triggering Monaco editor model updates (`editorRef.current.setValue(correctedCode)``), allowing immediate re-execution.

CHAPTER 6 — STUDENT PROFILE & EXTERNAL INTEGRATIONS

6.1 Student Identity & Connected Profile Architecture

Student identity management (``backend/app/models/profile.py``) is structured across two relational database models: ``StudentProfile`` (academic record, target role, target company, skills) and ``ConnectedProfile`` (external platform links for GitHub, LinkedIn, and LeetCode).

STUDENT PROFILE & EXTERNAL OAUTH INTEGRATION ARCHITECTURE

Central Student Identity (``users`` & ``student_profiles`` tables)

■■■ **Google OAuth 2.0 Integration:** ``/api/v1/auth/google`` (Verifies Google ID Token → Auto-creates User & Profile)

■■■ **GitHub Integration:** ``/api/v1/auth/github/connect`` (OAuth Token exchange → Fetches repos, stars, commit activity)

■■■ **LinkedIn Integration:** ``/api/v1/auth/linkedin/connect`` (OAuth 2.0 profile verification & handle linking)

■■■ **LeetCode Integration:** ``/api/v1/auth/leetcode/connect`` (Fetches solved problem stats, contest rating, submission counts)

↓ Persisted to SQLite ``connected_profiles`` Table (Unique Constraint: ``user_id`` + ``platform``)

Rendered in Frontend Workspace (``StudentProfile.tsx``) & Injected into Host Agent Context

Figure 6.1: Student Profile Architecture & External Platform Integrations.

6.2 Detailed External Platform Integrations

1. Google OAuth 2.0 Authentication (``auth.py``): Receives Google ID Tokens from Google Sign-In button on ``Login.tsx``. Verifies token signature via Google public keys, extracts email and name, and issues JWT access token.

2. GitHub OAuth & Repository Sync: Connects student GitHub identity. Fetches repository list, starred projects, top languages, and commit activity to present verified project credentials on the student dashboard.

3. LinkedIn Profile Verification: Authenticates candidate LinkedIn handles and stores public profile metadata to verify professional networking readiness.

4. LeetCode Stats Fetcher: Connects LeetCode handles and retrieves total solved count (Easy, Medium, Hard breakdown) to feed into the Placement Readiness Engine.

CHAPTER 7 — DASHBOARD, ANALYTICS & NOTIFICATIONS

7.1 Central Career Operating Dashboard (`Dashboard.tsx`)

The PlaceX Dashboard serves as the central mission control interface for students. Designed with a warm neutral aesthetic, it presents real-time placement readiness, active target roles, next milestone actions, module quick-launch cards, and recent host agent notifications.

7.2 Placement Readiness Scoring Model (`ReadinessEngine`)

The Placement Readiness Engine (`readiness\_engine.py`) computes an aggregate placement readiness score out of 100 based on multi-module performance:

$$\text{Readiness Score} = (\text{ATS Score} \times 0.40) + (\text{CodingSolvedPts} \times 0.35) + (\text{InterviewScore} \times 0.25)$$

Score Range	Readiness Tier Designation	System Guidance & Next Steps
0.0 – 39.9	Beginner Tier	Focus on foundational roadmap topics and initial resume upload.
40.0 – 59.9	Foundation Tier	Solve basic coding problems and address missing resume skills.
60.0 – 74.9	Intermediate Tier	Complete medium coding problems and attempt technical quizzes.
75.0 – 89.9	Interview Ready Tier	Conduct full AI mock placement interviews.
90.0 – 100.0	Placement Ready Tier	Fully prepared for top tier corporate campus placement rounds.

Table 7.1: Placement Readiness Tiers & Calculation Breakdown.

7.3 Real-Time Notification System (`notification\_service.py`)

The notification subsystem (`api/v1/notifications.py`) manages student alerts. Notifications are generated automatically when ATS analysis completes, coding problems are solved, or Host Agent recommendations update. Notifications are rendered in `Navbar.tsx` with read/unread badge counts.

CHAPTER 8 — FRONTEND, BACKEND & DATABASE INTEGRATION

8.1 Database Entity Relationship (ER) Schema (`placex.db`)

PlaceX utilizes a normalized relational database schema in SQLite (`placex.db`). The core tables related to author contributions include:

Table Name	Primary Key	Foreign Keys & Relationships	Core Columns / Purpose
`users`	`id` (Int)	None	`email`, `full_name`, `hashed_password`, `is_active`
`student_profiles`	`id` (Int)	`user_id` → `users.id` (1:1)	`target_role`, `target_company`, `degree`, `cgpa`, `profile_url`
`connected_profiles`	`id` (Int)	`user_id` → `users.id` (1:N)	`platform` (github/linkedin/leetcode), `profile_url`, `status`
`resume_uploads`	`id` (Int)	`user_id` → `users.id` (1:N)	`ats_score`, `raw_text`, `parsed_data`, `ats_breakdown`
`coding_submissions`	`id` (Int)	`user_id` → `users.id` (1:N)	`question_id`, `source_code`, `status`, `runtime_memory`
`student_roadmap_progress`	`id` (Int)	`user_id` → `users.id` (1:N)	`branch`, `level`, `completed_weeks`, `in_progress`
`host_agent_memory`	`id` (Int)	`user_id` → `users.id` (1:1)	`memory_data` (short_term, long_term, interaction_log)
`notifications`	`id` (Int)	`user_id` → `users.id` (1:N)	`title`, `message`, `type`, `is_read`, `created_at`

Table 8.1: PlaceX Relational Database Schema Specification (`placex.db`).

8.2 Security & Authentication Architecture

Security is implemented across three boundaries: (1) **Password Security:** Passwords hashed using bcrypt via Passlib; (2) **API Security:** OAuth 2.0 Password Bearer tokens signed with HS256 JWT encryption; (3) **CORS & Input Security:** Strict CORS origin filtering, Pydantic request payload validation, and server-side environment API key protection (`settings.GEMINI\_API\_KEY` is never exposed to client).

CHAPTER 9 — EMPIRICAL TESTING & EXPERIMENTAL RESULTS

9.1 Empirical Test Suite & Verification Matrix

To verify system correctness, an empirical test suite (`backend/scripts/`) was executed across all contributed modules. All 14 verification test cases passed with 100% success:

ID	Target Module	Test Input / Test Scenario	Expected System Output	Result
TC-01	ATS Engine	PDF Resume with complete contact details	Health Score ≥ 85.0 ; 5 breakdown scores generated	PASS
TC-02	ATS Matcher	Resume + Job Description (Python, JavaScript, SQL)	Match % calculated; Matched vs Missing skills split	PASS
TC-03	Coding Judge	Python code <code>print('Sum:', 10+20)</code>	Status: 'Accepted'; stdout: 'Sum: 30'; runtime recorded	PASS
TC-04	Coding Stdin	Python code <code>x=input()</code> with stdin <code>Hello</code>	Status: 'Accepted'; stdout contains <code>Hello</code>	PASS
TC-05	AST Analyzer	Nested <code>for</code> loop source code	Time Complexity estimated as $O(N^2)$; Complexity score calculated	PASS
TC-06	Host Agent	Request: 'Explain my code error' + System Error	System Agent returns root cause + corrected code replacement	PASS
TC-07	Host Agent	Request: 'Update target role to Data Scientist'	Tool <code>update_target_role</code> executed; DB updated successfully	PASS
TC-08	Profile OAuth	Google OAuth 2.0 ID Token payload	User authenticated; JWT token issued; Profile created	PASS
TC-09	GitHub Sync	Connect GitHub username 'testuser'	Public repos & commit stats fetched and saved in DB	PASS
TC-10	Notifications	Trigger ATS analysis completion event	Notification inserted into DB; unread count incremented	PASS
TC-11	Readiness	Resume score 80 + Coding 5 solved	Readiness Score calculated as 68.5 (Intermediate Tier)	PASS
TC-12	Roadmap DB	Query 288 DOCX roadmap weeks	All 12 recommendations return exactly 24 weeks each	PASS
TC-13	Vite Frontend	Production build command <code>npx vite build</code>	1000 modules transformed cleanly in 10.2s (0 errors)	PASS
TC-14	FastAPI Server	Health endpoint request <code>GET /health</code>	HTTP 200 OK; <code>status: healthy</code> , <code>database: connected</code>	PASS

Table 9.1: Empirical Verification Matrix across All Contributed Subsystems.

CHAPTER 10 — CONCLUSION & FUTURE ENGINEERING SCOPE

10.1 Engineering Summary & Contributions

This dissertation documented the design, implementation, and empirical verification of the core intelligence and developer tools of **PlaceX**. The author successfully architected: (1) A hypervisor-style Main Host Agent orchestrating multi-module context and tools; (2) A dual-mode ATS engine with mathematical health formulas and JD keyword matching; (3) An interactive multi-language Coding Sandbox with remote Judge0 execution, subprocess fallback with stdin linecache injection, and AST static complexity analysis; (4) A unified Student Profile engine with OAuth platform integrations; and (5) A centralized Dashboard with real-time notifications.

10.2 Future Engineering Scope

Future extensions include: (1) Expanding Judge0 languages to include Rust, Go, and TypeScript; (2) Integrating real-time WebSocket audio evaluation into the interview module; and (3) Migrating SQLite memory to PostgreSQL with vector embeddings (`pgvector`) for semantic similarity retrieval.

REFERENCES

[1] Google DeepMind. Gemini 1.5 & 2.0 API Technical Documentation & Function Calling Guide, 2024.

[2] FastAPI Project. High Performance Asynchronous Web Framework for Python, 2024.
<https://fastapi.tiangolo.com/>

[3] Microsoft. Monaco Editor Web SDK & Language Server Protocol Integration, 2024.
<https://microsoft.github.io/monaco-editor/>

[4] Judge0. Open Source Code Execution Engine Documentation, 2024. <https://judge0.com/>

[5] React Core Team. React 18 & Concurrent Rendering Architecture, 2024. <https://react.dev/>

[6] Python Software Foundation. Abstract Syntax Tree (``ast``) Module Reference, Python 3.12, 2024.

[7] Google Developers. Google Identity Services & OAuth 2.0 ID Token Verification, 2024.

[8] GitHub Developer REST API v3 Documentation, 2024. <https://docs.github.com/en/rest>

APPENDIX — CODEBASE TRACEABILITY MATRIX

To ensure 100% empirical traceability, the table below maps each thesis chapter directly to its source implementation files in the PlaceX repository:

Thesis Chapter	Primary Implementation File	Secondary Dependency File
Chapter 3 (Host Agent)	<code>`backend/app/host_agent/reasoning/orchestrator.py`</code>	<code>`backend/app/host_agent/context/context_engine.py`</code>
Chapter 3 (Agent Tools)	<code>`backend/app/host_agent/tools/tool_executor.py`</code>	<code>`backend/app/host_agent/tools/tool_registry.py`</code>
Chapter 3 (Agent API)	<code>`backend/app/api/v1/agent.py`</code>	<code>`frontend/src/components/chat/HostAgentWorkspace.tsx`</code>
Chapter 4 (ATS Engine)	<code>`backend/app/resume_service/ats/ats_engine.py`</code>	<code>`backend/app/resume_service/ats/matcher_engine.py`</code>
Chapter 4 (ATS UI)	<code>`backend/app/api/v1/resume.py`</code>	<code>`frontend/src/components/resume/ResumeAnalyzer.tsx`</code>
Chapter 5 (Coding Judge)	<code>`backend/app/coding_service/judge/judge0_adapter.py`</code>	<code>`backend/app/coding_service/services/coding_service.py`</code>
Chapter 5 (AST & UI)	<code>`backend/app/coding_service/analysis/ast_analyzer.py`</code>	<code>`frontend/src/components/coding/CodingSandbox.tsx`</code>
Chapter 6 (Profile OAuth)	<code>`backend/app/api/v1/auth.py`</code>	<code>`backend/app/models/profile.py`</code>

Chapter 6 (Profile UI)	`frontend/src/components/profile/StudentProfile.tsx`	`frontend/src/pages/Login.tsx`
Chapter 7 (Dashboard UI)	`frontend/src/pages/Dashboard.tsx`	`backend/app/services/notification_service.py`
Chapter 8 (Database DB)	`backend/app/database/session.py`	`backend/placex.db` (SQLite relational store)